

Module 7 — Implementation and Adoption

This is the implementation-and-adoption module of a knowledge base on production enterprise AI; it can be understood on its own.

Why this module exists: The hardest part of enterprise AI is usually not the technology. It's choosing the right problems, justifying the investment, redesigning how people work, governing it, and measuring whether it paid off. This module is the leadership and program view. It's deliberately less about tokens and more about turning capability into durable business value, and about avoiding the failure patterns that sink most initiatives.

This module covers six areas: use-case selection and ROI framing; build-versus-buy at the platform level; change management and workflow redesign; governance and maturity models; common enterprise failure patterns; and measuring value.

7.1 Use-Case Selection and ROI Framing

Why it matters. Most AI programs fail not because a model underperforms but because they picked the wrong use case. Too risky, too vague, too low-value, or a poor fit for what LLMs are actually good at. Disciplined selection is the highest-leverage decision in the entire program.

How it works — selection criteria. Score candidate use cases on several dimensions.

First, value. That means hard dollars — cost reduction, revenue, cycle-time — or strategic value. It has to be big enough to matter.

Second, feasibility and AI-fit. Does it play to LLM strengths, like language, summarization, extraction, drafting, retrieval-grounded question answering, and classification? Or does it lean on weaknesses, like exact calculation without tools, guaranteed correctness, or real-time precision?

Third, data readiness. Is the grounding data available, clean, accessible, and permissioned? This is frequently the real gate.

Fourth, risk and stakes. Consider the consequences of error, reversibility, and regulatory exposure. High stakes does not mean avoid, but it also does not mean start here.

Fifth, feedback loop. Can you measure quality and improve it with evaluations?

And sixth, adoption feasibility. Will users actually change their behavior?

A useful two-by-two plots value against risk and effort. Start in the high-value, low-risk quadrant: internal productivity, drafting, retrieval question answering, and deflection. Graduate to high-value, high-risk work

— like autonomous actions on the system of record — once you've built trust, evaluations, and governance.

Now, the sobering data on why selection discipline matters. MIT's 2025 report, "The GenAI Divide: State of AI in Business," found that despite an estimated thirty to forty billion dollars in enterprise GenAI spend, roughly 95 percent of organizations reported no measurable profit-and-loss return. It attributed the failure to the integration or "learning gap" — tools that don't retain feedback or fit workflows — and not to model quality. One flag here: this measures pilots lacking measurable ROI, not technical failure, and on a modest sample. Cite it as "no measurable ROI," not "failed." McKinsey's 2025 State of AI similarly found only about 39 percent report any enterprise EBIT impact, with most attributing under five percent — the "scaling gap." The lesson is that value comes from disciplined selection plus workflow integration, not from the model.

Framing the ROI. Quantify the baseline — time, cost, and error today — then the expected improvement, and then the full cost. Full cost means model and API spend, infrastructure, build, ongoing evaluation and maintenance, and change management. Include the often-forgotten run costs: prompt and model spend at scale, human-in-the-loop labor, and maintenance as models and data evolve. Frame ROI as a portfolio, with some quick wins to fund and justify the harder bets, rather than a single moonshot.

When to use it (and when not to). Prefer use cases where AI augments a human — drafting, assisting, recommending — over full automation for the first wave. That's lower risk, faster value, and it builds trust and training data. Favor problems with abundant, accessible, well-governed data. Kill or defer use cases that require guaranteed correctness without a verification path.

Where it goes wrong. Watch for a solution in search of a problem, where "we need an AI strategy" leads to building tech rather than solving a problem. Watch for boiling the ocean — one huge transformational project instead of compounding wins. Watch for ignoring data readiness, where the demo works on cherry-picked data but the production data isn't ready. Watch for chasing the flashiest, riskiest use case first, like an autonomous agent on financial writes, before the organization has evaluations, governance, and trust. And watch for having no ROI baseline, which means you can't prove value later.

A concrete example. A business-to-business company starts with support-ticket deflection and rep-assist. That's high value, low risk, data ready, and measurable. It does this rather than building a fully autonomous quoting agent. The early wins fund and de-risk the later, higher-stakes configure-price-quote agent initiative. By the time it gets there, the company already has evaluations, governance, and organizational trust.

7.2 Build-Versus-Buy at the Platform Level

Why it matters. Enterprises must decide, layer by layer, what to build, what to buy, and what to adopt from a platform they already own. Getting this wrong wastes money by rebuilding commodities, or it creates lock-in and gaps by buying where you needed control. The calculus is unusual here because the underlying tech moves so fast that today's custom build is tomorrow's commodity feature.

How it works — the spectrum. Buy or consume the commoditizing layers: foundation models via API or cloud, embeddings, vector databases, observability, and guardrail libraries. Building these rarely makes sense.

Adopt platform-native where the use case lives inside a system of record you already run. Examples include Salesforce Agentforce, ServiceNow AI Agents, Microsoft Copilot, and cloud agent services. The pros are deep integration, inherited security and governance, and fast time-to-value. The cons are lock-in, walled-garden limits, cost, and less flexibility across an estate with multiple systems of record.

Build or assemble the differentiating middle: your retrieval quality, your semantic layer and knowledge, your orchestration and guardrails, and your evaluations. These are the parts that encode competitive advantage and control. This is usually assembly of bought components, not building from scratch.

And within "buy," there's model choice: API versus open or self-hosted, driven by residency, control, and cost.

When to use it (and when not to). Buy commodities. Adopt platform-native for single-platform, system-of-record-embedded workflows where speed and built-in governance win. Build or assemble where differentiation, control, cross-platform reach, or vendor-neutrality matter. This "buy or partner over build" lean is data-backed: MIT in 2025 found that vendor and partnership deployments succeeded roughly twice as often as internal builds — about 67 percent versus about one third. Key questions to ask: Is this a commodity or a differentiator? Does it need to span multiple systems of record? What's the lock-in and switching cost? Can we evaluate and govern a bought solution to our standard? Preserve optionality with abstractions like a model gateway, MCP, and a semantic layer.

Where it goes wrong. Watch for building commodities, like a homegrown vector database or a bespoke model, which becomes sunk cost and is quickly outdated. Watch for over-buying into one platform and then hitting its ceiling or lock-in across a fragmented estate. Watch for ignoring the total cost and lock-in of platform-native licensing at scale. Watch for reinventing governance the platform already provides — or, conversely, assuming a composable stack "inherits" governance that it doesn't. And watch for framework and tool churn turning a "build" into perpetual rework.

A concrete example. A firm running both Salesforce and ServiceNow adopts Agentforce for CRM-native selling assist and Now Assist and AI Agents for service workflows. That's platform-native, fast, and

governed. But it builds a composable orchestration layer with MCP for cross-platform journeys that must read from both systems of record plus an ERP, because no single platform owns the whole process. The rule of thumb: walled-garden where it's self-contained, vendor-neutral where the estate is fragmented.

7.3 Change Management and Workflow Redesign

Why it matters. Technology adoption is a people problem. The largest AI ROI comes not from bolting a copilot onto an unchanged process, but from redesigning the workflow around what AI now makes cheap — and bringing people along. Ignore this and you get expensive shelfware with 5 percent adoption. BCG's long-standing framing quantifies it: roughly 70 percent of AI value comes from people and process change, about 20 percent from tech and data, and about 10 percent from the algorithms. That's a reused heuristic, not a fresh survey, so cite it as BCG's framing. BCG's 2025 work also found only about 5 percent of firms are "future-built" value generators, while about 60 percent are laggards.

How it works. Start with workflow redesign. Don't pave the cow path. Ask: if drafting, summarizing, retrieval, and triage are now near-free, how should this process work? Redesign roles and hand-offs so that AI drafts and a human reviews and decides.

Next, trust-building. Start with augmentation, where the human stays in control. Show reliability with citations and explainability, and let evidence, not mandate, drive adoption.

Then training and enablement. Teach people to prompt, to verify AI output rather than blindly trust it — that's the automation-bias trap — and to know when not to use it.

Then incentives and metrics. Align individual incentives with adoption. Measure and celebrate wins. Make the AI-assisted path the easy default by embedding it in existing tools and system-of-record surfaces, not a separate app.

Then feedback channels. Users report failures, and those become evaluation cases and improvements — a virtuous loop that also builds ownership.

And finally, job and role impact. Address fears honestly. Frame the change as augmentation and reskilling, and be transparent about what's changing.

When to use it (and when not to). Embed AI where work already happens — inside the CRM or the IT service management tool, not as a bolt-on chatbot — to minimize behavior change. Sequence it like this: augmentation first, then trust, then workflow redesign, then expanded autonomy. Invest in enablement in proportion to the behavior change required.

Where it goes wrong. Watch for "build it and they will come," with no adoption plan, so the tool is ignored. Watch for paving the cow path, automating a broken process instead of redesigning it. Watch for

over-trust and automation bias, where users rubber-stamp wrong outputs — or the opposite, under-trust, where an early visible failure poisons the well. Watch for separate-app friction, forcing users out of their system of record into a standalone tool. And watch for ignoring the human impact, which breeds resistance, fear, and quiet non-adoption.

A concrete example. Instead of a standalone quoting chatbot, the configure-price-quote assistant lives inside the CRM where reps already work. It drafts quotes for rep review, cites the pricing rules it used, and gets better as reps correct it. Adoption climbs because it fits the existing workflow and visibly saves time. And the redesigned process — AI drafts, rep approves, system of record records it — is faster and better-governed than before.

7.4 Governance and Maturity Models

Why it matters. AI governance is the organizational system — policies, roles, review processes, standards — that ensures AI is built and used responsibly, safely, and in compliance. Maturity models describe the stages organizations move through, giving a roadmap and a way to assess where you are. Together they turn ad-hoc experimentation into a scalable, trustworthy capability.

How it works. Start with the governance components. You need an AI governance body that's cross-functional, spanning legal, security, data, business, and ethics. You need an acceptable-use policy. You need a use-case intake and risk-tiering process, with controls proportionate to risk. You need standards for evaluation, security, data handling, and human oversight. You need an AI system inventory, vendor and model risk assessment, and incident response for AI failures. And you need alignment to frameworks like the NIST AI Risk Management Framework, ISO/IEC 42001, and EU AI Act obligations.

Then the operating model. It ranges across a spectrum. At one end is centralized: a central AI team or platform, which gives control and consistency but can bottleneck. In the middle is federated, or hub-and-spoke: a central platform plus standards, with business units building on top, which scales and is common at maturity. At the other end is decentralized: fast, but risky and inconsistent. Most mature organizations land on hub-and-spoke, where a central platform or center of excellence provides paved-road tooling, guardrails, and governance, and teams innovate within it. Industry data suggests firms that successfully scale AI are disproportionately likely to use hub-and-spoke. The frequently-cited "roughly three times more likely" figure traces to consultancy analyses rather than a named primary survey, so treat it as directional. And governance is lagging adoption: Deloitte's 2025 State of AI in the Enterprise found only about 21 percent of organizations have a mature governance model for agentic AI — agentic AI is scaling faster than guardrails.

Then the maturity stages, typically four. Stage one is experimentation: scattered pilots and shadow AI. Stage two is foundational: first production use cases, basic governance, and a platform emerging. Stage three is operational or scaling: repeatable delivery, evaluations, hub-and-spoke, and reuse. And stage four is

transformational: AI reshapes core processes, agents work inside workflows, and strong governance and measurement are embedded.

When to use it (and when not to). Stand up lightweight governance early — risk-tiering plus basic standards — enough to be safe without smothering experimentation. Then mature toward hub-and-spoke with a paved road, meaning a shared platform, guardrails, and evaluations, as use cases multiply. Match control intensity to the risk tier, not a blanket policy.

Where it goes wrong. Watch for no governance at all, which invites shadow AI, data leakage, and inconsistent, risky deployments. Watch for governance so heavy it kills innovation, where every use case is a six-month review and teams route around it. Watch for governance without enablement — rules but no paved road, so teams can't comply easily. Watch for perpetual pilots that never cross from experimentation to production, the "pilot purgatory" failure. And watch for no central learning, where every team re-solves retrieval, evaluation, and guardrails from scratch.

A concrete example. A company forms an AI governance board and adopts risk-tiering. Tier one, internal drafting, gets light controls. Tier three, customer-facing writes to the system of record, gets full evaluations, human-in-the-loop, and audit. It stands up a central platform team providing a paved road: approved models via a gateway, shared retrieval, guardrail, and evaluation tooling, and MCP connectors to Salesforce and ServiceNow. Business units ship fast on that road. That's hub-and-spoke maturity.

7.5 Common Enterprise Failure Patterns

Why it matters. Enterprise AI failures rhyme. Naming the patterns lets teams recognize and avoid them. This section is a checklist that the video series can turn into a memorable "how not to" episode.

How it works — the pattern catalog. First, pilot purgatory, or "death by proof of concept." Endless demos that never reach production because there's no path through security, governance, integration, and evaluations. The fix: pick production-viable use cases and build the operational path from day one.

Second, a solution in search of a problem. Tech-first, not value-first.

Third, data-readiness denial. The demo works, but production data is fragmented, stale, or ungoverned. This is the most common silent killer.

Fourth, fine-tuning to inject knowledge. It's stale and hallucination-prone; you should have used retrieval-augmented generation instead.

Fifth, no evaluation. You can't tell if it's good, can't catch regressions, and can't improve. You're flying blind.

Sixth, hallucination reaching users. No grounding or verification on factual or high-stakes outputs.

Seventh, entitlement or security bypass. A service-account god-mode, or retrieval that ignores sharing rules. That's a breach, and a deployment blocker.

Eighth, cost blowups. A frontier model everywhere, deep agent loops, and no caching or budgets.

Ninth, over-automation too soon. Autonomous high-stakes actions before trust, guardrails, and evaluations exist. Real cases make this concrete: Klarna publicly reversed its AI-replaces-agents strategy in 2025 and rehired humans after quality dropped. McDonald's ended its AI drive-thru pilot. And Taco Bell scaled its back after edge-case failures — the "eighteen thousand waters" incident. This is the demo-to-production gap made real.

Tenth, agent-washing and premature autonomy hype. Gartner in 2025 projects that more than 40 percent of agentic AI projects will be canceled by the end of 2027 on cost, value, and risk grounds. It warns of "agent washing" — rebranded chatbots and robotic process automation sold as agents. Buy real capability, not the label, and pilot with a value and risk gate.

Eleventh, adoption failure. Great tech, no change management, and it becomes shelfware.

Twelfth, multi-agent over-engineering. Complexity where one agent or workflow would do.

Thirteenth, framework churn and lock-in. Betting the architecture on a fast-moving framework, or on a single platform across an estate with multiple systems of record.

Fourteenth, governance extremes: none, which is chaos, or too much, which is paralysis.

And fifteenth, ignoring maintenance. Treating AI as build-once, when models and data drift, prompts rot, and fine-tunes go stale.

When to use it (and when not to). Run new initiatives against this checklist as a pre-mortem. Most fixes trace back to earlier practices: value-first selection, data readiness, evaluations, grounding, entitlements, cost control, human-in-the-loop, and change management.

A concrete example. A retailer's "AI concierge" stalls in pilot purgatory. Impressive demo, but no entitlement model, no evaluations, and data spread across three systems. The reset: narrow to one high-value flow, build the data backbone plus entitlement-aware retrieval plus an evaluation harness first, ship an augmentation-mode version, then expand. The technology barely changed. The approach did.

7.6 Measuring Value

Why it matters. If you can't measure value, you can't sustain investment, prioritize, or improve. Measurement closes the loop from the ROI framing discussed earlier — and it's a common blind spot. Teams ship and never quantify impact, then struggle to justify the next phase.

How it works — the metric layers. Start with system and quality metrics: accuracy, groundedness, task-completion, guardrail triggers, and human-intervention rate. These are necessary but not sufficient — they aren't business value.

Then operational metrics: latency, cost per task, throughput, and deflection or automation rate.

Then business outcome metrics, which are the point: cycle-time reduction, such as quote turnaround; cost savings, such as support cost per ticket; revenue impact, such as conversion, upsell, and win rate; quality and error reduction; customer satisfaction and net promoter score; and employee productivity and satisfaction.

Then adoption metrics: active users, usage depth, and retention. A great tool nobody uses has no value.

Then attribution. Baseline before, measure after, and where possible use controlled comparisons like A/B tests or holdouts to isolate AI's contribution from confounds.

And keep in mind leading versus lagging indicators. Adoption and quality are leading indicators; business outcomes lag. Track both.

When to use it (and when not to). Define success metrics and a baseline before building. Tie system metrics to a business outcome, and don't optimize a proxy. Use holdouts or A/B tests where feasible for credible attribution. Report a balanced scorecard covering quality, operations, business, and adoption — not a single vanity number.

Where it goes wrong. Watch for no baseline, which makes impact unprovable. Watch for vanity metrics, like "queries served" without outcomes. Watch for proxy optimization, with great offline scores but no business impact. Watch for ignoring adoption, so value is assumed rather than realized. Watch for over-attribution, crediting AI for gains driven by other changes because there was no holdout. And watch for only measuring cost savings, which misses revenue, quality, and experience value — or the reverse.

A worked ROI calculation — so that "quantify it" isn't hand-waving. Take a support-deflection use case. The annual value equals tickets per month, times twelve months, times the deflection rate, times the fully-loaded cost per ticket, minus the annual run cost. Now plug in the numbers: fifty thousand tickets a month, times twelve months, times a thirty percent deflection rate, times eight dollars a ticket, equals 1.44 million dollars gross. Minus 400,000 dollars of run cost — that's model spend plus human-in-the-loop review plus maintenance — is about 1.04 million dollars net a year. The discipline is less the exact number than forcing every term to be explicit, especially the run costs teams forget, which ties back to the full-cost point from earlier.

The attribution problem is real, and a clean A/B test is often infeasible. You frequently can't run a clean per-transaction A/B test for high-stakes writes to the system of record. You can't randomize the same rep, and deal-level randomization has tiny samples and confounds like deal size and rep skill. Practical

alternatives include rep-cohort or region holdouts, matched-pair analysis, or a staggered rollout with difference-in-differences, which compares the change in treated versus not-yet-treated groups. Name the confounds explicitly. A credible directional estimate beats a falsely-precise one.

A concrete example. For the configure-price-quote assistant, the team baselines quote turnaround at an average of two days, plus quote error rate and win rate. Because a per-deal A/B test isn't clean, they use a region-holdout and staggered rollout. They see turnaround drop to about four hours, quote errors down meaningfully, and a modest win-rate lift on faster quotes, plus about 80 percent rep adoption. These figures are illustrative, not benchmarks. That balanced, attributable scorecard justifies expanding from augmentation toward more autonomous quoting, with governance in place.

Module 7 Takeaways

Select use cases on value, feasibility, data-readiness, risk, and measurability. Start with high-value, low-risk augmentation, and graduate to higher-stakes autonomy.

On build versus buy: buy commodities, adopt platform-native for system-of-record-embedded single-platform work, and build or assemble the differentiating middle. Preserve optionality with a gateway, MCP, and a semantic layer.

Change management and workflow redesign is where the big ROI is. Embed AI where work happens, augment before automating, redesign the process, and build trust with evidence.

On governance and maturity: go lightweight early, then move toward a hub-and-spoke paved road. Keep control risk-tiered and proportionate, and avoid both chaos and paralysis.

Know the failure patterns — pilot purgatory, data-readiness denial, no evaluations, entitlement bypass, cost blowups, over-automation, and adoption failure — and pre-mortem against them.

And measure value with a baseline and a balanced scorecard covering quality, operations, business, and adoption, attributed via holdouts where possible.